

CS223 Computer Architecture & Organization

Superscalar Processors



John Jose

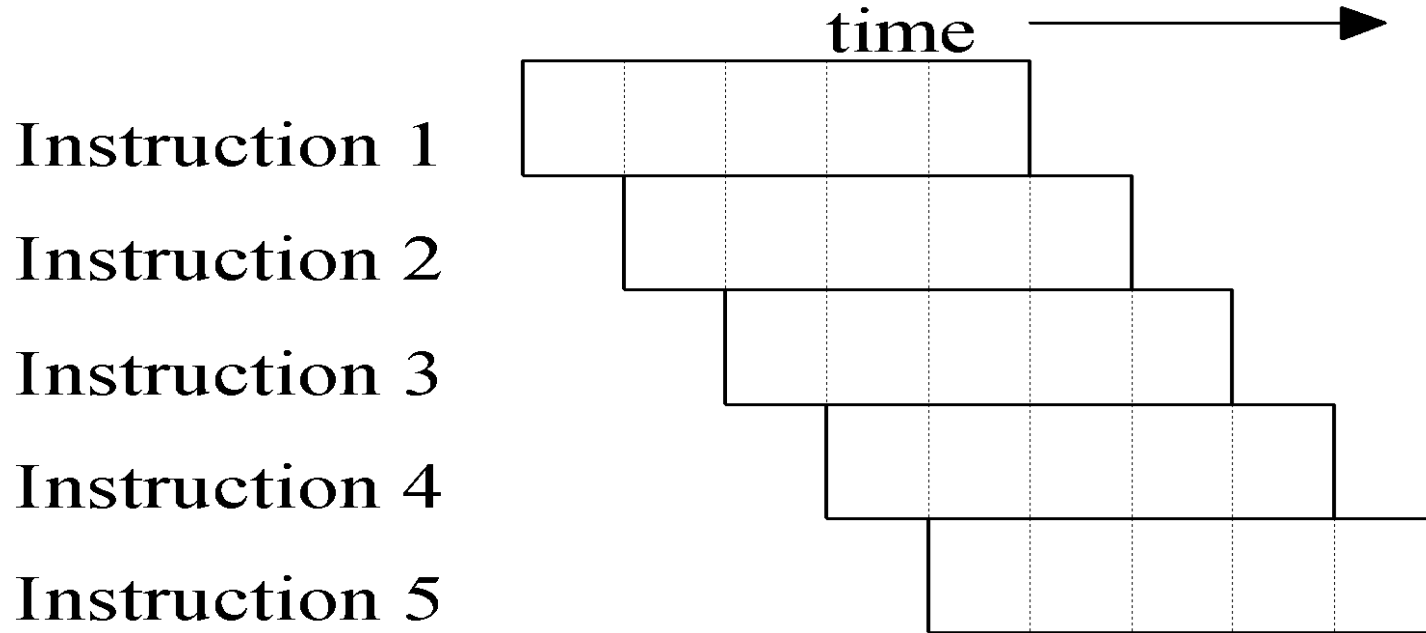
Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Simple in-order pipelining

Pipelining –goal was to complete one instruction per clock cycle



Advanced Pipelining

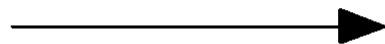
- ❖ Increase the depth of the pipeline to increase the clock rate – **superpipelining**
- ❖ Fetch (and execute) more than one instructions at one time (expand every pipeline stage to accommodate multiple instructions) – **multiple-issue (super scalar)**
- ❖ Launching multiple instructions per stage allows the instruction execution rate, CPI, to be less than 1

Superpipelining

❖ **superpipelining** - Increase the depth of the pipeline to

increase the clock rate

time



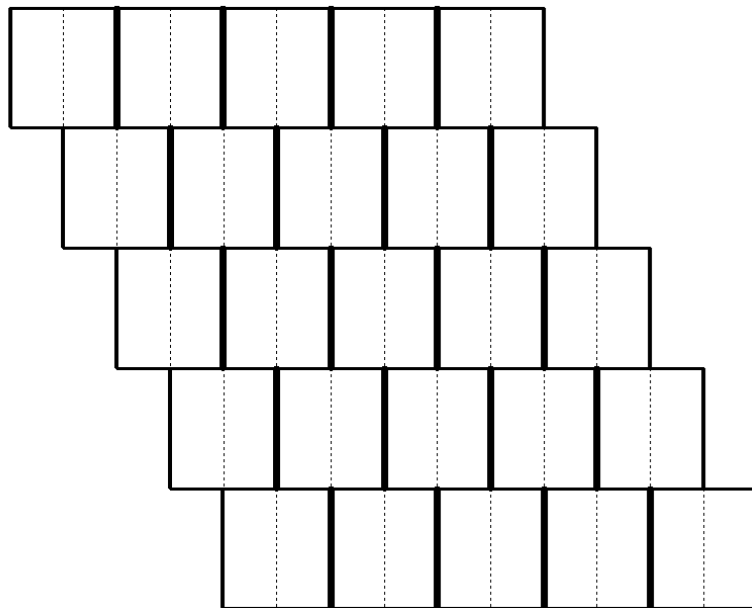
Instruction 1

Instruction 2

Instruction 3

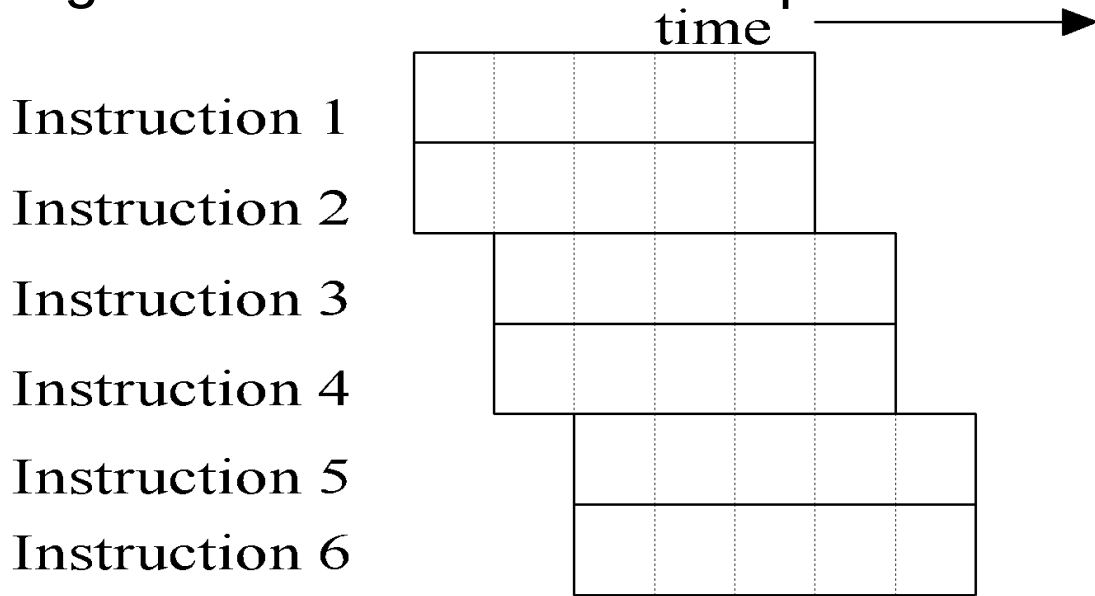
Instruction 4

Instruction 5



Superscalar – multiple-issue

- ❖ Fetch (and execute) more than 1 instructions at one time
- ❖ Expand every stage to accommodate multiple instructions



Multiple Issue and Static Scheduling

- ❖ Basic pipeline will give only a min CPI of 1
- ❖ To achieve $CPI < 1$, need to complete multiple instructions per clock (**superscalar processors**)
- ❖ Multiple functional units and Multiple Issue
- ❖ Solutions:
 - ❖ Statically scheduled superscalar processors -VLIW
VLIW (very long instruction word) processors
 - ❖ Dynamically scheduled superscalar processors

VLIW Processors

- ❖ Package multiple operations into one instruction (long word)
- ❖ The instruction have a set of fields for each functional unit.
- ❖ 16 to 24 bits per unit, → instruction length of 80 to 120 bits.
- ❖ Must be enough parallelism in code to fill the available slots
- ❖ Find enough parallel instructions by loop unrolling and scheduling

VLIW Processors

❖ Example VLIW processor:

- ❖ One integer instruction (or branch)
- ❖ Two independent floating-point operations
- ❖ Two independent memory references

				<u>Clock cycle issued</u>
Loop:	L.D	F0,0(R1)		1
	<i>stall</i>			2
	ADD.D	F4,F0,F2		3
	<i>stall</i>			4
	<i>stall</i>			5
	S.D	F4,0(R1)		6
	DADDUI	R1,R1,#-8		7
	<i>stall</i>			8
	BNE	R1,R2,Loop		9

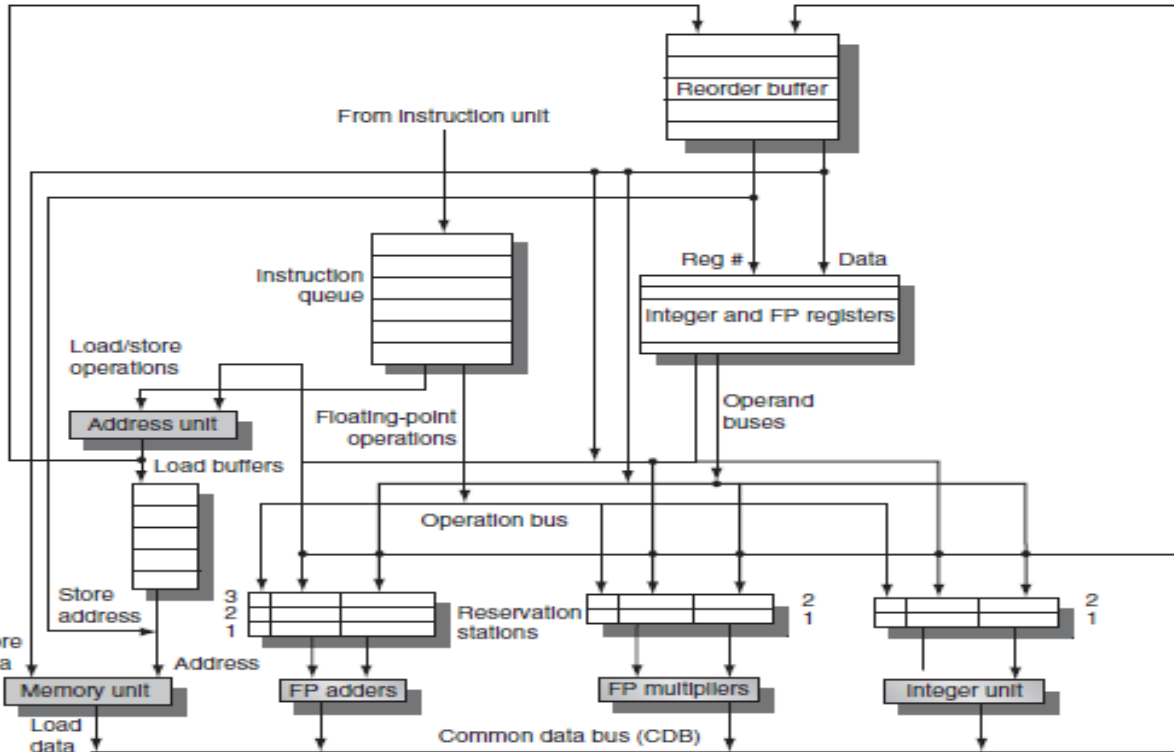
Memory reference 1	Memory reference 2	FP operation 1	FP operation 2	Integer operation/branch
L.D F0,0(R1)	L.D F6,-8(R1)			
L.D F10,-16(R1)	L.D F14,-24(R1)			
L.D F18,-32(R1)	L.D F22,-40(R1)	ADD.D F4,F0,F2	ADD.D F8,F6,F2	
L.D F26,-48(R1)		ADD.D F12,F10,F2	ADD.D F16,F14,F2	
		ADD.D F20,F18,F2	ADD.D F24,F22,F2	
S.D F4,0(R1)	S.D F8,-8(R1)	ADD.D F28,F26,F2		
S.D F12,-16(R1)	S.D F16,-24(R1)			DADDUI R1,R1,#-56
S.D F20,24(R1)	S.D F24,16(R1)			
S.D F28,8(R1)				BNE R1,R2,Loop

❖ Performance: 9 cycles to complete 26 instructions. More registers used.

Extreme Optimization

- ❖ Modern micro-architectures uses this triple combination:
 - ❖ Dynamic scheduling + multiple issue + speculation
- ❖ Dependency between instructions issued in same clock.
- ❖ Register read for multiple instructions in parallel.
- ❖ Complex Issue logic to check dependencies and hazards.
- ❖ Assign reservation stations and ROB entries in a cycle.

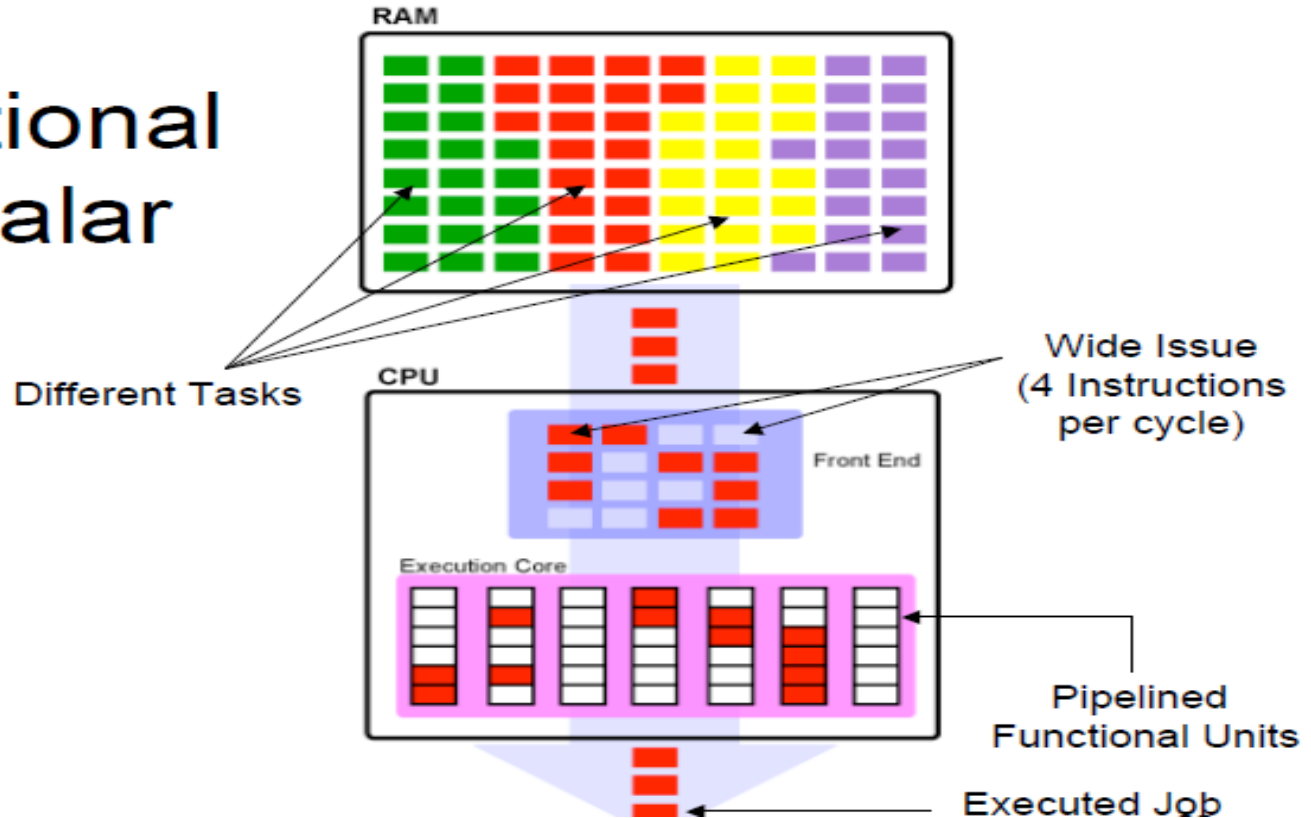
Handling Multiple Issue



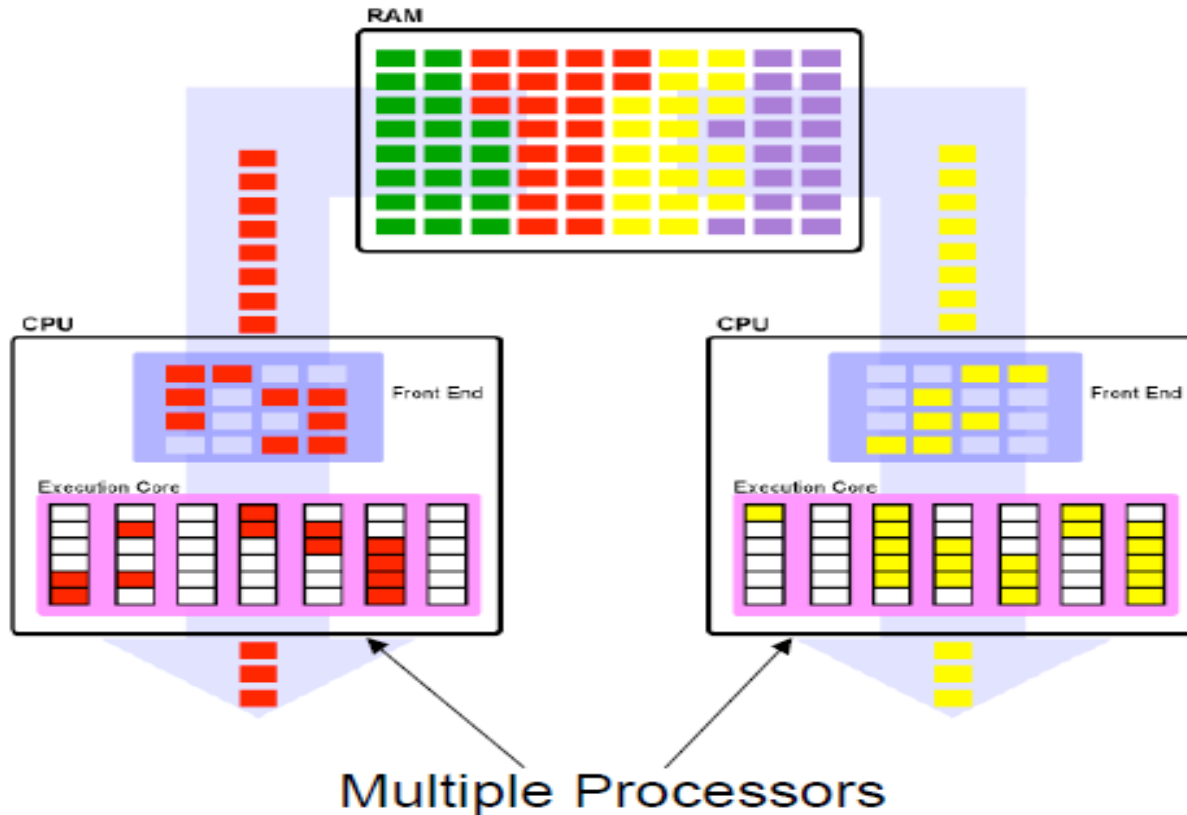
- ❖ One issue per unit per cycle
- ❖ Wider operand buses, CDB
- ❖ Multiple register read/cycle
- ❖ Multiple instruction to commit / cycle

Superscalar Processor

Conventional Superscalar

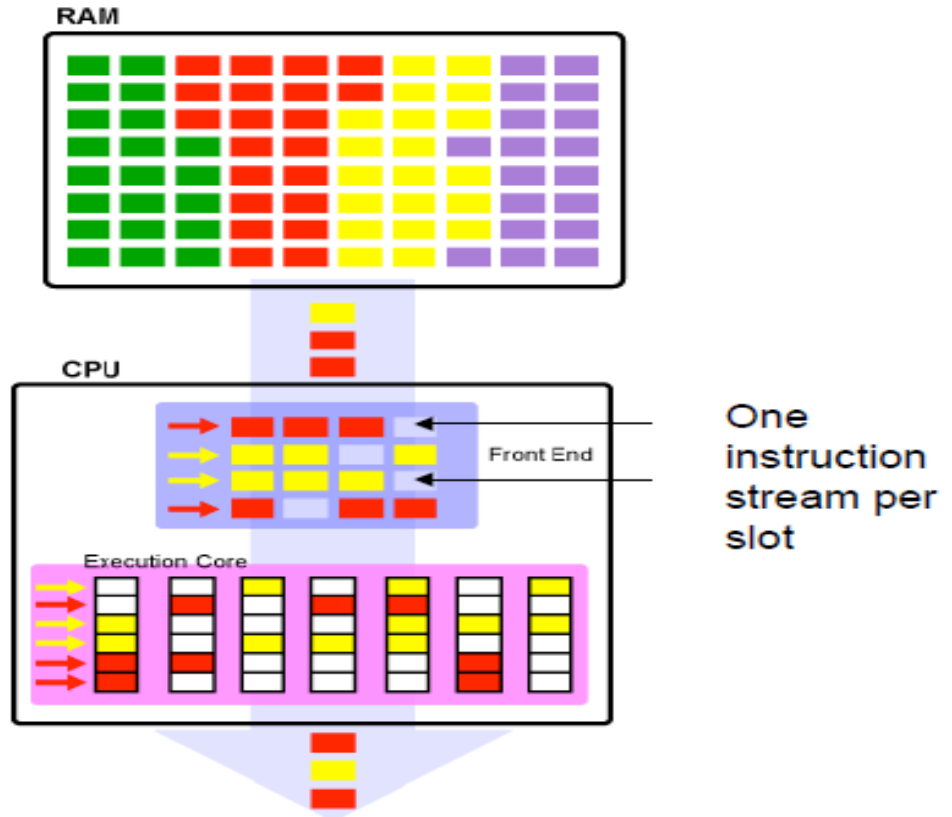


Symmetric Multiprocessor



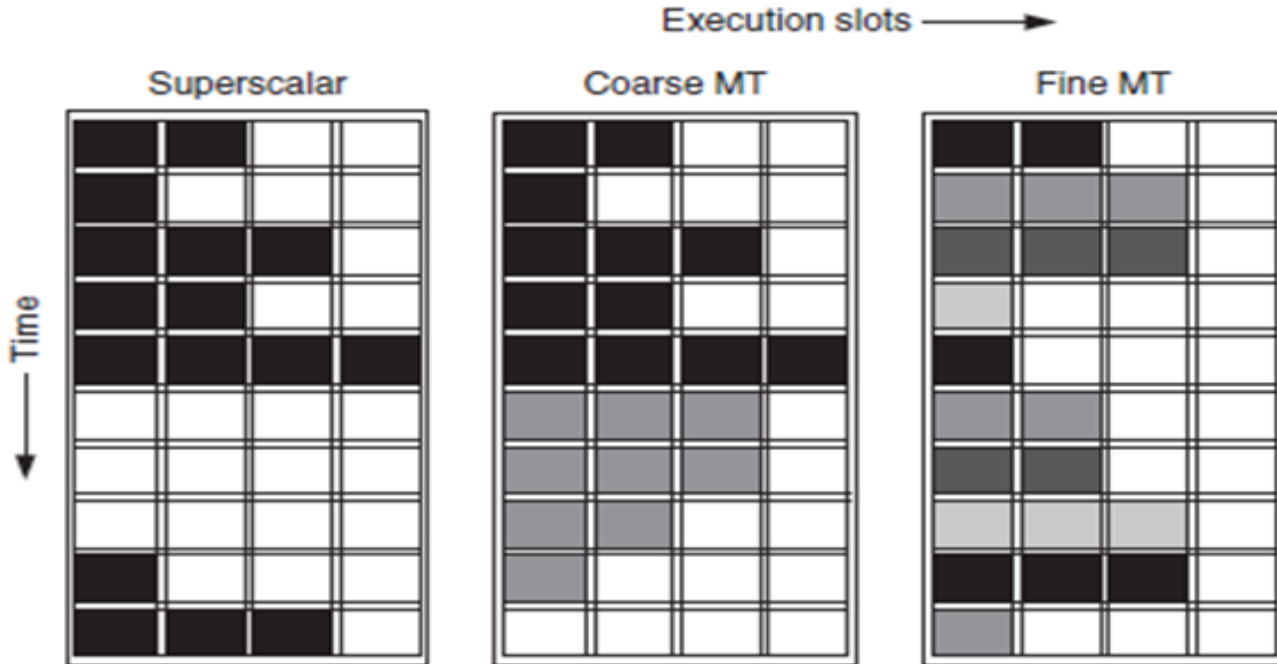
Multithreading

Multithreading



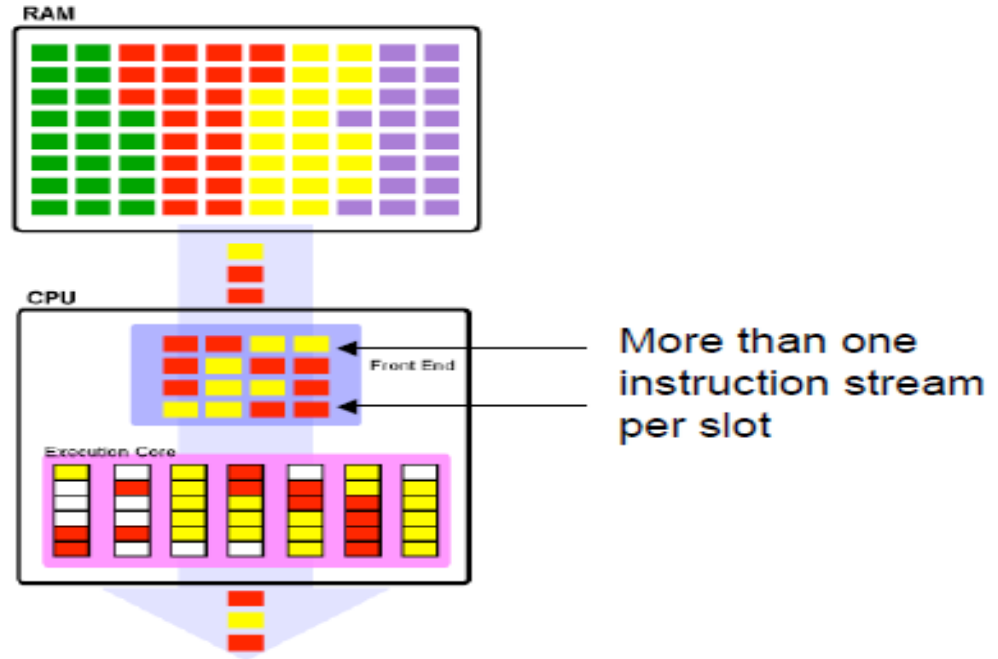
Multithreading

Multithreading allows multiple threads to share the functional units of a single processor in an overlapping fashion

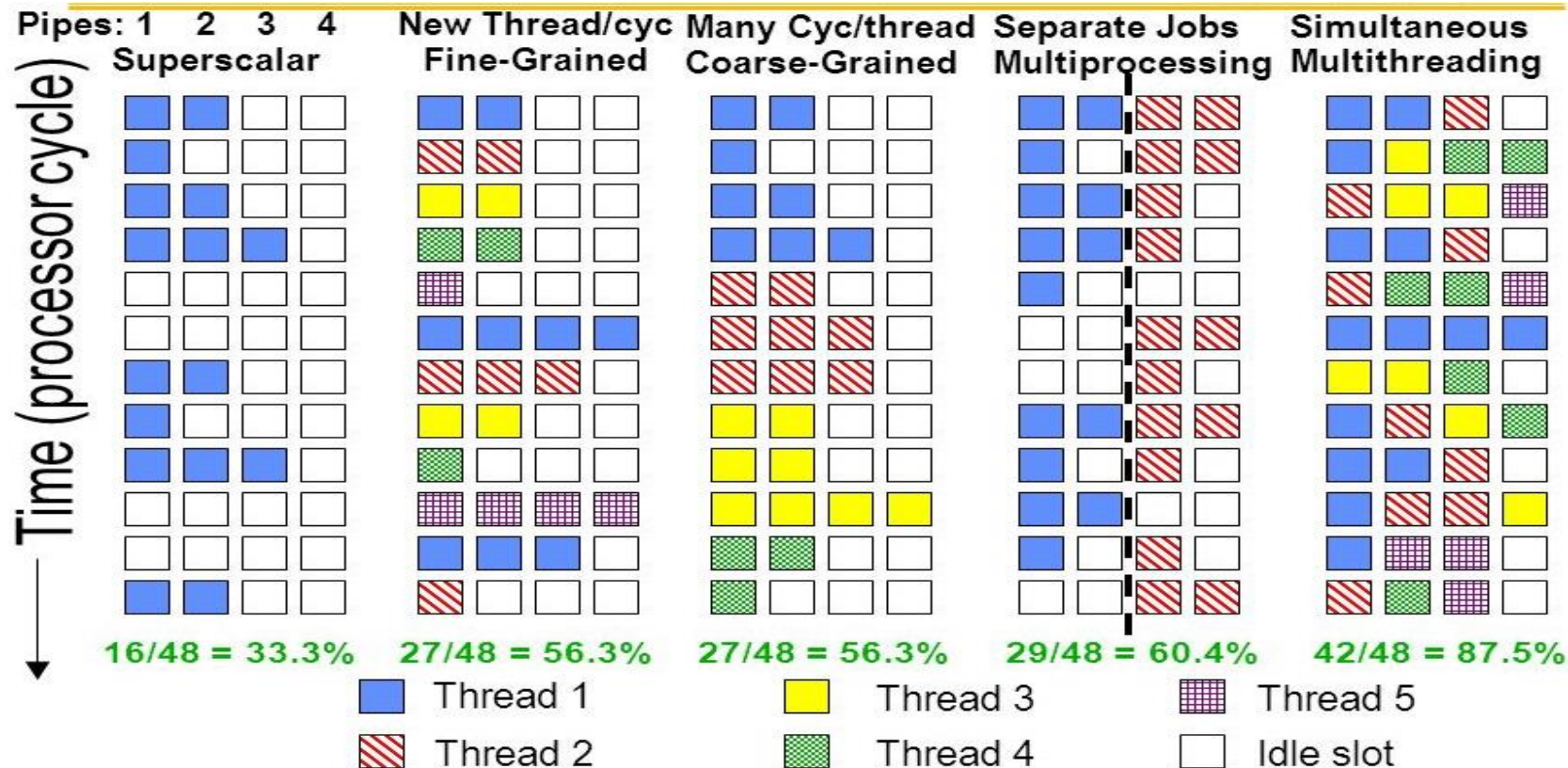


Hyperthreading /SMT

Hyperthreading



Comparison of Resource Utilization



Cache optimization for superscalar



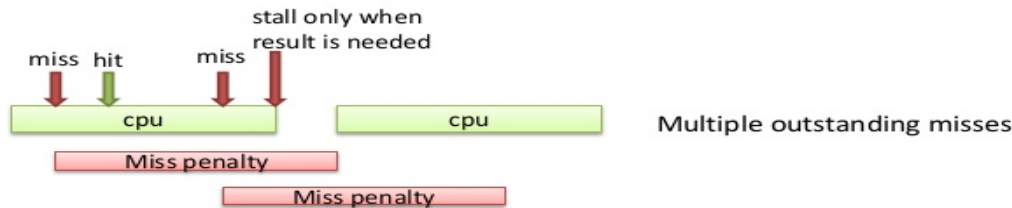
Average memory access time = Hit time + (Miss rate $\times$ Miss penalty)

Pipelining Cache

- ❖ **Pipelined cache for faster clock cycle time.**
- ❖ Split cache memory access into several sub-stages
- ❖ Ex: Indexing, Tag Read, Hit/Miss Check, Data Transfer
- ❖ Pipeline cache access to improve bandwidth
 - ❖ Examples: Pentium 4 – Core i7: 4 cycles
- ❖ But slow in hits.
- ❖ Increases branch mis-prediction penalty
- ❖ Makes it easier to increase associativity

Non-blocking Caches

- ❖ **Non blocking caches to increase cache bandwidth**
- ❖ Caches can serve hits under multiple cache miss in progress
 - ❖ *(a) Hit under miss (b) Hit under multiple miss*



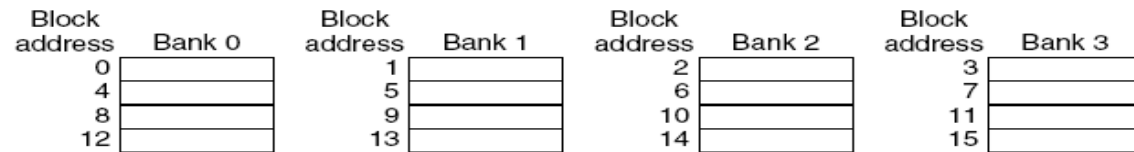
- ❖ Must needed for OOO superscalar processor for IPC increase
- ❖ L2 must support it with L1-MSHR (Miss Status Holding Reg.)
- ❖ On an L1 miss allocate MSHR entry, clear upon L2 respond with cache block reply.

Non-blocking Caches

- ❖ **Non blocking caches to increase cache bandwidth**
- ❖ Processors can hide L1 miss penalty but not L2 miss penalty
- ❖ Reduces the effective miss penalty by overlapping miss latencies
- ❖ Significantly increases the complexity of the cache controller as there can be multiple outstanding memory accesses
- ❖ Requires pipelined or banked memory system

Multi-banked Caches

- ❖ **Multi-banked caches to increase cache bandwidth**
- ❖ Rather having a single monolithic block, divide cache into independent banks that can support simultaneous accesses.
 - ❖ ARM Cortex-A8 supports 1-4 banks for L2
 - ❖ Intel i7 supports 4 banks for L1 and 8 banks for L2
- ❖ Sequential Interleaving



Four-way interleaved cache banks using block addressing. Assuming 64 bytes per blocks, each of these addresses would be multiplied by 64 to get byte addressing.

Early Restart

- ❖ **Early restart to reduce miss penalty**
- ❖ Do not wait for entire block to be loaded for restarting CPU
- ❖ **Early restart**
 - ❖ Request words in normal order
 - ❖ Send missed work to the processor as soon as it arrives
 - ❖ L2 controller is not involved in this technique

- **Early restart**



1 -> 2 -> 3 -> 4

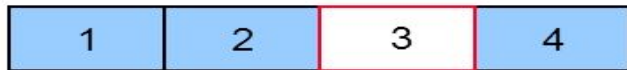
Requested word: word 3



Processing performed
background

Critical Word First

- ❖ **Critical word first to reduce miss penalty**
- ❖ **Critical word first**
 - ❖ Request missed word from memory first
 - ❖ Send it to the processor as soon as it arrives
 - ❖ Processor resume while rest of the block is filled in cache
 - ❖ L2 cache controller forward words of a block out of order.
 - ❖ L1 cache controller should re-arrange words in block
 - **Critical word first**

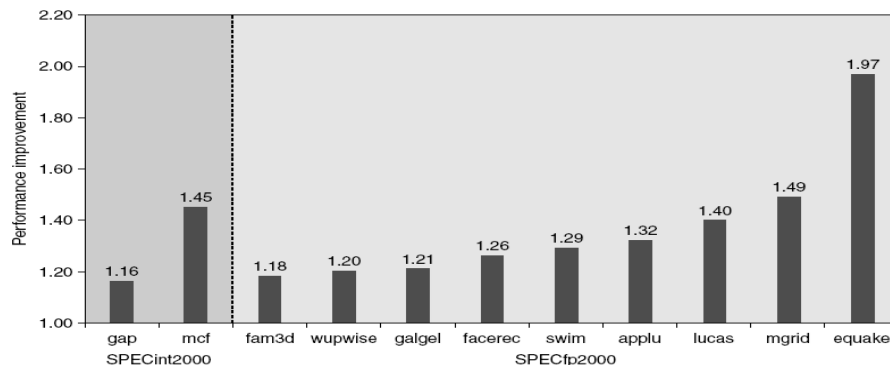
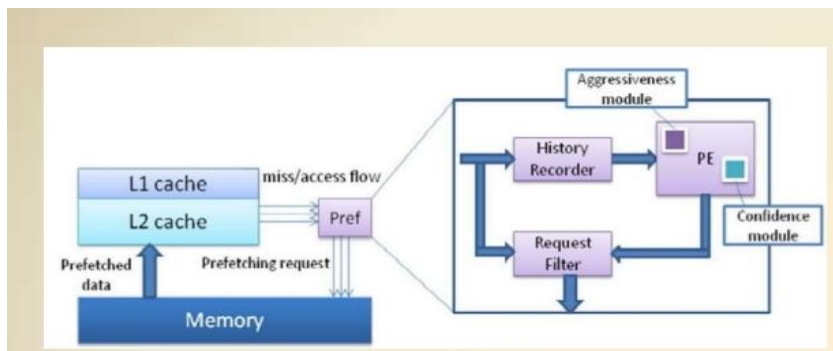


3 -> 4 -> 1 -> 2

As soon as 3 is brought it is forwarded to CPU
In background 4, 1 and 2 is brought

Hardware Prefetching

- ❖ **Pre-fetching to reduce miss rate and miss penalty.**
- ❖ Pre-fetch items before processor request them.
- ❖ Fetch more blocks on miss -include next sequential block
- ❖ Requested block is kept in I-cache and next in stream buffer.
- ❖ If a missed block is in stream buffer, cache miss is cancelled





johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>